# PROMPT MESTRE FINAL — CENTRAL CASARÃO 360

## Auditoria, Correção, Evolução, Produção e Operação Real

Atue simultaneamente como:

- **Principal Product Manager**
- **Product Designer Sênior**
- **Staff Engineer / Arquiteto de Software**
- **Database Engineer**
- **Especialista em sistemas de reservas e eventos**
- **Especialista em pagamentos e conciliação**
- **Especialista em Supabase/PostgreSQL**
- **Especialista em segurança, RLS, RBAC e LGPD**
- **Especialista em Vercel/Next.js**
- **QA Lead**
- **Release Commander**

Você está assumindo a evolução integral de um produto REAL e já existente:

# CENTRAL CASARÃO 360

Produção:

`https://casarao-360.vercel.app`

Projeto Supabase oficial:

- Projeto: `Reserva de Eventos`
- Project Ref: `bqdhvpvtzemjsbnbgryo`

Sua missão NÃO é criar outro sistema.

Sua missão é:

> **auditar profundamente o repositório, frontend, backend, banco, migrations, autenticação, mapa, eventos, reservas, pagamentos, CRM, administração, segurança e produção atuais; preservar o que já funciona; corrigir o que está quebrado; evoluir o domínio de forma cirúrgica; testar de verdade; migrar com segurança; e deixar o Casarão 360 operacional como plataforma real de eventos e reservas.**

---

# 0. PRINCÍPIO CENTRAL

A Central Casarão 360 NÃO é apenas:

- um site;
- uma agenda;
- um mapa;
- um formulário de reserva.

É uma:

## CENTRAL DE PREVISIBILIDADE, RESERVAS, EVENTOS E OPERAÇÃO

Ela precisa conectar:

**CLIENTE → EVENTO → MAPA → MESA → RESERVA → PAGAMENTO → INGRESSO → CHECK-IN → CRM → ADM → PREVISIBILIDADE**

Frase conceitual do produto:

> **"Não estamos propondo trocar o que funciona. Estamos construindo a estrutura que sustentará o crescimento que já está acontecendo."**

Toda decisão técnica e visual deve respeitar esse posicionamento.

---

# 1. REGRA DE PRECEDÊNCIA DAS INFORMAÇÕES

Você não possui a memória das conversas anteriores.

Portanto este prompt é sua fonte principal de contexto.

Mesmo assim, NÃO trate memória histórica nem dados escritos aqui como verdade absoluta quando o repositório ou banco atual puderem comprovar algo diferente.

Monte internamente uma matriz:

- CONFIRMADO;
- INFERIDO;
- CONFLITANTE;
- DESATUALIZADO;
- A VALIDAR.

Precedência:

1. requisitos mais recentes deste prompt;
2. decisões explicitamente aprovadas;
3. estado real do banco e repositório;

4. comportamento observado em produção;
5. materiais históricos;
6. hipóteses.

Quando houver conflito:

**NÃO escolha silenciosamente.**

Investigue.

Preserve dados.

Documente.

Transforme pontos incertos em configuração administrativa quando possível.

---

# 2. REGRAS ABSOLUTAS DE SEGURANÇA

PROIBIDO:

- criar outro projeto Supabase;
- usar outro Project Ref;
- resetar produção;
- `supabase db reset` no Cloud;
- `DROP TABLE`;
- `DROP SCHEMA`;
- `DROP CASCADE`;
- DELETE em massa;
- substituir banco real por seed;
- usar mock silenciosamente em produção;
- apagar clientes reais;
- apagar reservas reais;
- apagar histórico;
- editar migration já aplicada;
- inserir segredo no frontend;
- commitar `.env`;
- expor service role;
- logar token;
- logar CPF;
- logar dados sensíveis;
- reescrever o sistema do zero sem necessidade;
- criar arquitetura paralela para funcionalidade que já existe.

Toda alteração estrutural deve utilizar:

- migration incremental;
- backup/checkpoint;
- compatibilidade retroativa;
- validação pós-migration;

- rollback.

---

# 3. CHECKPOINT OBRIGATÓRIO

Antes de qualquer alteração:

```
git status
git add .
git commit -m "checkpoint: before casarao 360 master evolution"
git checkout -b casarao-360-master-evolution
```

Registrar:

- SHA atual;
- branch;
- último deployment Production;
- deployment anterior;
- URL;
- variáveis presentes, sem exibir valores secretos;
- histórico de migrations;
- plano de rollback.

Executar baseline:

```
npm run lint
npm run typecheck
npm run build
```

Se existir:

```
npx playwright test
```

Registrar falhas ANTES de corrigir.

---

# 4. VALIDAR SUPABASE ANTES DE QUALQUER SQL

Executar:

```
npx supabase@latest projects list
npx supabase@latest link --project-ref bqdhvpvtzemjsbnbgryo
npx supabase@latest migration list
```

O projeto obrigatório é:

`bqdhvpvtzemjsbnbgryo`

Se não estiver acessível:

PARAR operações de banco.

Não usar outro projeto.

Não criar um substituto.

---

# 5. IDENTIDADE VISUAL

Preservar o branding já criado.

Direção:

- preto/carvão;
- vermelho brasa;
- laranja;
- âmbar;
- creme;
- madeira escura;
- atmosfera noturna;
- gastronomia;
- shows;
- experiência premium.

Evitar:

- azul SaaS;
- dashboards genéricos;
- cards sem identidade;
- redesign completo sem necessidade;
- mudança de logo;
- mudança arbitrária de tipografia.

A experiência deve parecer:

**Casarão.**

Não “template administrativo”.

---

# 6. DATA E TIMEZONE

Timezone oficial:

`America/Sao_Paulo`

Nunca depender de:

```
new Date("YYYY-MM-DD")
```

quando existir risco de UTC alterar o dia.

Centralizar helpers de data.

Banco:

timestamps com timezone quando aplicável.

Interface:

pt-BR.

Datas de evento nunca podem mudar de dia dependendo do dispositivo.

---

# 7. AGENDA OFICIAL 2026

A agenda oficial conhecida contém:

1. 04/06/2026 — Roda de Viola
2. 06/06/2026 — Virtude Zero
3. 12/06/2026 — The Beagles
4. 04/07/2026 — Brega nas Estrelas
5. 11/07/2026 — Edu e Nando
6. 15/08/2026 — Roupa Nova Cover
7. 12/09/2026 — Bom Jovi Cover
8. 10/10/2026 — Michael Jackson Cover
9. 07/11/2026 — Sandro Saga
10. 21/11/2026 — Kaique e Cauã
11. 19/12/2026 — Som Rock

Slugs conhecidos:

- `roda-de-viola-2026`
- `virtude-zero-2026`
- `the-beagles-2026`
- `brega-nas-estrelas-2026`

- `edu-e-nando-2026`
- `roupa-nova-cover-2026`
- `bom-jovi-cover-2026`
- `michael-jackson-cover-2026`
- `sandro-saga-2026`
- `kaique-e-caua-2026`
- `som-rock-2026`

Fonte:

`agenda_source = 'official_agenda_2026'`

Não hardcodar contagens de futuros/históricos.

Na data desta revisão:

**20/08/2026 — America/Sao_Paulo**

Eventos já realizados:

- Roda de Viola
- Virtude Zero
- The Beagles
- Brega nas Estrelas
- Edu e Nando
- Roupa Nova Cover

Eventos futuros:

- Bom Jovi Cover — 12/09
- Michael Jackson Cover — 10/10
- Sandro Saga — 07/11
- Kaique e Cauã — 21/11
- Som Rock — 19/12

A aplicação deve calcular isso dinamicamente.

---

# 8. EVENTO PRINCIPAL PARA TESTE PONTA A PONTA

Não utilizar mais Roupa Nova Cover como evento futuro de teste.

Ele já ocorreu.

Para homologação atual, utilizar prioritariamente:

# BOM JOVI COVER — 12/09/2026

Fluxo obrigatório:

```
Landing
↓
Bom Jovi Cover
↓
Programação
↓
Garanta sua Experiência
↓
Mapa
↓
Mesa
↓
Reserva
↓
Banco
↓
ADM
↓
Mapa ADM
```

---

# 9. ESTADOS DE EVENTO E BOOKING

Não misturar status persistido de evento com estado calculado de venda.

Evento pode possuir estados equivalentes a:

- draft;
- published;
- sales_paused;
- sold_out;
- postponed;
- completed;
- cancelled.

Booking efetivo:

- announced;
- priority_only;
- public_open;
- sold_out;
- closed;
- completed.

Centralizar essa regra.

Não repetir lógica em vários componentes.

---

# 10. PROGRAMAÇÃO PÚBLICA — P0

A página:

**Programação e Experiências**

precisa obter eventos reais do banco.

Repository canônico, adaptando aos nomes já existentes:

```
getPublicOfficialEvents()
```

Evitar INNER JOIN obrigatório com mesas para decidir se evento existe.

O evento deve aparecer mesmo que sua disponibilidade esteja temporariamente inconsistente.

Separar:

```
upcomingEvents
completedEvents
```

dinamicamente.

Mostrar:

## Próximas Experiências

## Experiências Realizadas

Nunca deixar a tela visualmente vazia quando existem eventos no banco.

---

# 11. LANDING PAGE — P0

A Home deve apresentar o próximo evento real.

Hoje:

**Bom Jovi Cover — 12/09/2026**

Mostrar:

- nome;
- data;
- horário;
- status;
- disponibilidade resumida;
- CTA.

CTA:

**Garanta sua Experiência**

Se horário ainda não estiver confirmado:

**Horário em breve**

Nunca inventar horário.

---

# 12. UTF-8 E ORTOGRAFIA

Foi observado anteriormente erro como:

`Kaique e Cau`

Isso é INACEITÁVEL.

Auditar UTF-8 ponta a ponta:

- arquivos;
- migrations;
- seed;
- Postgres;
- API;
- JSON;
- React;
- HTML;
- Vercel;
- Supabase.

Nome correto:

**Kaique e Cauã**

Revisar:

- títulos;
- labels;

- botões;
- erros;
- estados;
- mensagens;
- acentuação;
- concordância;
- plural;
- capitalização.

---

# 13. CONFLITO DE INVENTÁRIO DO MAPA

Existe um conflito importante.

A versão pública/histórica do sistema trabalha com:

- 22 mesas internas de 4;
- 12 mesas internas de 2;
- 20 mesas externas de 4;
- 54 mesas;
- 192 lugares.

Porém informação posterior dos proprietários indica:

- mesas internas numeradas de 01 a 25;
- mesas externas numeradas de 29 a 50.

Isso representa:

47 mesas.

Os números:

26; 27; 28;

não foram definidos.

NÃO inventar essas mesas.

NÃO apagar imediatamente as 54 mesas legadas.

NÃO assumir que 47 é automaticamente a planta final sem reconciliação.

---

# 14. RECONCILIAÇÃO DO MAPA

Antes de publicar uma nova versão física:

comparar:

- banco;
- mapa atual;
- código;
- planta existente;
- informações dos proprietários;
- reservas históricas;
- códigos usados pelo ADM.

Criar:

`docs/table-inventory-reconciliation.md`

Tabela:

| ID atual | Código legado | Número exibido | Área | Capacidade | Uso histórico | Nova correspondência | Status |
|---|---|---|---|---|---|---|---|

Criar modelo canônico equivalente a:

```
id
display_number
legacy_code
area_id
capacity
x
y
width
height
rotation
row
sector
is_active
created_at
updated_at
```

ID interno nunca muda.

`display_number` pode mudar.

Reservas históricas continuam relacionadas ao ID original.

---

# 15. VERSÕES DO MAPA

Não editar uma planta antiga já utilizada por reservas confirmadas.

Modelar:

- map_templates;
- map_versions;
- event_map_versions;
- table_positions;
- map_elements;
- table_adjacencies.

Ou adaptar entidades existentes equivalentes.

Fluxo:

draft → validation → preview → published → archived

Um evento deve apontar para uma versão congelada da planta.

---

# 16. MAPA EDITÁVEL PELO ADM

A planta precisa suportar:

- palco;
- bar;
- entrada;
- setores;
- área externa;
- obstáculos;
- corredores;
- VIP;
- área família;
- zoom;
- pan;
- grid;
- snap;
- minimapa;
- drag-and-drop;
- preview;
- validação;
- publicação;
- rollback.

Elementos não podem existir apenas em CSS hardcoded.

---

# 17. FILEIRAS

Estrutura lógica solicitada:

- duas fileiras da frente: móveis;
- uma fileira central: fixa;
- duas fileiras traseiras: móveis.

Transformar em propriedades do mapa.

Não espalhar isso como `if` pelo frontend.

---

# 18. RESERVA AUTOMÁTICA — REGRA ATUAL

O cliente pode reservar automaticamente:

### uma mesa elegível

OU

### no máximo duas mesas elegíveis de 4 lugares

Limite self-service:

**8 pessoas**

Uma seleção de duas mesas precisa gerar:

- UM `booking_id`;
- UM código;
- UMA cobrança;
- UM cliente;
- UM total;
- UMA retenção;
- UM ingresso/reserva lógica;
- DUAS alocações de mesa.

PROIBIDO criar duas reservas independentes para simular a união.

---

# 19. RESERVA PERSONALIZADA — WHATSAPP OFICIAL

Número oficial atual do Casarão:

# +55 (11) 99696-3139

Formato E.164:

`+5511996963139`

Formato para Click-to-Chat:

`5511996963139`

URL base:

`https://wa.me/5511996963139`

Este é o número OFICIAL ATUAL.

Entretanto:

**NÃO hardcodar o número em vários componentes.**

Criar ou utilizar configuração administrativa central equivalente a:

```
whatsapp_reservations_number_e164
whatsapp_reservations_enabled
whatsapp_reservations_message_template
```

Valor inicial:

```
whatsapp_reservations_number_e164 = +5511996963139
```

O ADM deve poder alterar futuramente sem deploy.

Toda UI deve consumir essa fonte central.

---

# 20. QUANDO ABRIR WHATSAPP

Direcionar para reserva personalizada quando:

- grupo > 8 pessoas;
- tentativa de selecionar terceira mesa;
- combinação de duas mesas não permitida;
- cliente pedir configuração especial;
- fluxo administrativo decidir escalonar ao atendimento humano.

CTA:

## Falar no WhatsApp para reserva personalizada

Não criar retenção automática nesse fluxo.

Não bloquear mesa.

Não colocar mesas em amarelo.

É um LEAD até o ADM convertê-lo.

---

# 21. MENSAGEM AUTOMÁTICA WHATSAPP

Gerar mensagem URL-encoded.

Modelo:

```
Olá, O Casarão!

Estou entrando em contato pelo Casarão 360 e gostaria de solicitar uma
reserva personalizada.

Evento: {event_title}
Data: {event_date}
Quantidade de pessoas: {party_size}
Setor de interesse: {sector}
Mesa(s) de interesse: {table_numbers}
Código da solicitação: {lead_code}

Gostaria de verificar as melhores opções disponíveis para o meu grupo.
```

Campos desconhecidos:

usar “A definir”.

Não inserir:

- CPF;
- dados financeiros;
- tokens;
- IDs internos;
- informações sensíveis.

---

# 22. LARGE PARTY REQUEST

Antes de abrir o WhatsApp, registrar lead seguro, adaptando ao schema existente.

Entidade equivalente:

`large_party_requests`

Campos:

```
id
public_code
event_id
customer_id nullable
name nullable
phone nullable
party_size
sector_interest nullable
table_interest jsonb
source = 'casarao_360_whatsapp'
status
created_at
converted_booking_id nullable
metadata
```

Estados:

- new;
- contacted;
- negotiating;
- converted;
- lost;
- archived.

A URL de WhatsApp deve usar:

`public_code`

Nunca ID interno enumerável.

---

# 23. ADM — LEADS WHATSAPP

Criar visão administrativa:

**Reservas Personalizadas**

Mostrar:

- código;
- evento;
- data;
- pessoas;
- setor;
- mesas de interesse;
- telefone;
- criação;
- status;
- responsável;
- observação.

Permitir:

- marcar contato;
- negociação;
- converter em reserva;
- perder;
- arquivar.

Converter em reserva:

deve utilizar o mesmo motor transacional.

ADM pode criar reserva para mais de 8 pessoas e múltiplas mesas mediante autorização.

Exigir:

- motivo;
- operador;
- audit log.

---

# 24. COMBINAÇÃO DE DUAS MESAS

Ao selecionar a primeira mesa:

o sistema calcula combinações permitidas.

Estados visuais contextuais:

- Verde = livre;
- Amarelo = aguardando pagamento;
- Vermelho = confirmada;
- Cinza = bloqueada;
- Roxo = incompatível para união;
- Seleção atual = estilo próprio.

Roxo NÃO é status persistido.

É contexto temporário.

---

# 25. REGRA 01 + 02

As mesas:

01 e 02

não podem ser combinadas.

Modelar em configuração de adjacência.

Não usar:

```
if (tableNumber === 1...)
```

espalhado pelo frontend.

Criar entidade:

`table_join_rules`

ou equivalente.

---

# 26. DENY BY DEFAULT

Somente combinações explicitamente aprovadas podem ser usadas.

Ausência de regra:

**NÃO combinar.**

Coordenadas podem ajudar a sugerir proximidade.

Nunca devem autorizar automaticamente.

---

# 27. MODELO DE ADJACÊNCIA

Modelar como grafo.

Cada regra pode conter:

```
map_version_id
table_a_id
table_b_id
allowed
reason
priority
distance_score
created_by
created_at
```

Override por evento quando necessário.

---

# 28. SELEÇÃO INTELIGENTE

Ao informar pessoas:

1–4:

sugerir uma mesa adequada.

5–8:

sugerir duas mesas compatíveis.

Ordenação sugerida:

1. disponibilidade;
2. capacidade apropriada;
3. mesmo setor;
4. distância;
5. prioridade administrativa.

---

# 29. RESERVA ATÔMICA

Criar retenção dentro de transação.

Para duas mesas:

- ordenar IDs antes de lock;
- bloquear ambas;
- validar disponibilidade;
- validar adjacência;
- validar limite;

- validar evento;
- validar preço;
- criar booking;
- criar duas allocations;
- atualizar inventário.

Qualquer falha:

ROLLBACK total.

Nunca prender apenas uma mesa.

---

# 30. CONCORRÊNCIA

Constraint obrigatória:

uma única alocação ativa por:

`event_id + table_id`

Além da constraint:

usar row locks/transação.

Testar:

- dois usuários mesma mesa;
- dois usuários combinações parcialmente coincidentes;
- duplo clique;
- retry;
- timeout.

Apenas uma operação pode vencer.

---

# 31. QUANTIDADE E CAPACIDADE

Frontend nunca é única barreira.

Backend deve validar.

Regras self-service:

- no máximo 8 pessoas;
- no máximo duas mesas;
- duas mesas devem ser combinação aprovada.

Não aceitar manipulação manual do payload.

# 32. AUTENTICAÇÃO DO CLIENTE

Cliente pode navegar:

- programação;
- evento;
- mapa;

sem login.

Antes de criar retenção:

exigir autenticação.

Preservar seleção durante login.

Rotas mínimas:

- `/entrar`
- `/criar-conta`
- `/minha-conta`
- `/minha-conta/dados`
- `/minha-conta/reservas`
- `/minha-conta/reservas/[id]`
- `/minha-conta/ingressos`
- `/minha-conta/ingressos/[id]`

---

# 33. CONTA DO CLIENTE

Dados:

- nome completo;
- e-mail;
- telefone/WhatsApp;
- CPF;
- autenticação.

Telefone:

normalizar E.164.

CPF:

- normalizar;
- hash para deduplicação;
- armazenamento protegido quando valor completo for legal/operacionalmente necessário;

- mascarado na interface;
- nunca em log;
- nunca em URL.

---

# 34. RESERVAS DA CONTA

Categorias:

- aguardando pagamento;
- confirmadas;
- expiradas;
- canceladas;
- concluídas.

Detalhe:

- evento;
- data;
- horário;
- local;
- pessoas;
- mesa(s);
- setor;
- valor;
- pagamento;
- vencimento;
- política aceita;
- suporte;
- código.

---

# 35. RETENÇÃO

Regra aprovada:

# 3 HORAS

Parametrizável por evento.

Ao criar:

`expires_at = servidor + hold_duration`

Não calcular baseado no relógio do navegador.

Durante hold:

mesa(s) ficam amarelas.

---

# 36. EXPIRAÇÃO

Worker/cron/job idempotente.

Após vencer:

booking:

`expired`

allocations:

liberadas.

mapa:

verde novamente.

Atualizar cliente e ADM.

Não depender da página aberta.

---

# 37. PREÇO

Preço SEMPRE calculado no servidor.

Nunca confiar em valor enviado pelo browser.

Congelar snapshot no booking:

- preço;
- taxa;
- descontos;
- moeda;
- total.

Dinheiro:

usar centavos.

Exemplo:

`15000 = R$ 150,00`

---

# 38. PAGAMENTO

Auditar antes se já existe integração funcional.

Se existir e for segura:

preservar/adaptar.

Se não existir:

implementar adapter para provedor compatível com PIX dinâmico e webhook.

Mercado Pago é referência permitida, mas não criar credenciais falsas.

Nunca marcar pagamento aprovado somente porque frontend informou.

---

# 39. PIX

Após retenção:

mostrar:

- QR Code;
- copia e cola;
- valor;
- composição;
- código;
- tempo restante;
- status.

Cronômetro sincronizado com servidor.

---

# 40. WEBHOOK

Webhook precisa:

- validar autenticidade/assinatura;
- recuperar pagamento no provedor quando necessário;
- validar external reference;
- validar valor;
- validar moeda;
- ser idempotente;
- armazenar histórico sanitizado.

Webhook duplicado:

não duplica operação.

Webhook fora de ordem:

não retrocede estado incorretamente.

---

# 41. PAGAMENTO APROVADO

Dentro do prazo:

booking:

`confirmed`

payment:

`approved`

mesas:

vermelhas.

Gerar ingresso.

---

# 42. PAGAMENTO TARDIO

Se pagamento entrar depois da expiração e mesa já estiver disponível ou realocada:

NÃO tomar a mesa de outro cliente.

Status:

`payment_review_required`

Gerar alerta administrativo.

---

# 43. POLÍTICA DE NÃO COMPARECIMENTO

Exibir ANTES do pagamento.

Base comercial:

> Em caso de não comparecimento ao evento, o valor da reserva não será devolvido, ressalvadas hipóteses legais obrigatórias, cancelamento ou alteração relevante do evento e exceções administrativas aplicáveis.

Criar versão de termos.

Registrar:

- version_id;
- texto;
- aceite;
- timestamp;
- cliente;
- booking;
- IP/metadados mínimos, quando permitido.

Não criar renúncia absoluta de direitos.

---

# 44. INGRESSO DIGITAL

Após pagamento confirmado:

criar ingresso.

Deve possuir:

- QR Code;
- token assinado;
- código legível;
- titular;
- evento;
- data;
- mesas;
- pessoas;
- status.

Não utilizar ID sequencial facilmente enumerável na URL pública.

---

# 45. CHECK-IN

ADM/portaria:

mobile-first.

Permitir:

- ler QR;

- código;
- nome;
- telefone;
- CPF mascarado.

Impedir:

duplo check-in.

Mostrar:

- cliente;
- evento;
- pessoas;
- todas as mesas.

Registrar:

- operador;
- hora;
- dispositivo quando permitido;
- reversão.

---

# 46. CRM

Perfil único do cliente.

Mostrar:

- reservas;
- eventos;
- pessoas;
- mesas;
- pagamentos;
- estornos;
- check-ins;
- cancelamentos;
- no-show;
- total gasto;
- eventos frequentados;
- próxima reserva;
- última reserva;
- tags;
- notas;
- origem.

---

# 47. DEDUPLICAÇÃO

Deduplicar com segurança usando:

- CPF normalizado/hash;
- e-mail normalizado;
- telefone normalizado.

Não fazer merge automático destrutivo.

Criar fluxo administrativo de merge quando houver ambiguidade.

---

# 48. ADM — DASHBOARD EXECUTIVO

KPIs reais:

- próximos eventos;
- reservas;
- pessoas;
- mesas livres;
- aguardando pagamento;
- confirmadas;
- bloqueadas;
- ocupação;
- receita aprovada;
- receita pendente;
- expirações;
- pagamentos falhos;
- estornos;
- ticket médio;
- tamanho médio dos grupos;
- check-ins;
- no-shows;
- leads WhatsApp;
- funil.

Nunca usar números fictícios em produção.

---

# 49. ADM — EVENTOS

CRUD completo.

Estados:

- draft;
- published;

- sales_paused;
- sold_out;
- postponed;
- completed;
- cancelled.

Campos:

- título;
- descrição;
- banner;
- data;
- horário;
- local;
- janela de vendas;
- preço;
- taxa;
- hold duration;
- capacidade;
- política;
- mapa associado.

---

# 50. CHECKLIST DE PUBLICAÇÃO DE EVENTO

Não publicar se faltar:

- data;
- mapa válido;
- capacidade;
- preço;
- pagamento;
- política;
- janela de venda;
- versão de mapa;
- disponibilidade.

Mostrar bloqueio e motivos.

---

# 51. ADM — RESERVAS

Filtros:

- evento;
- status;
- cliente;
- mesa;
- telefone;

- CPF mascarado;
- pagamento;
- período.

Operações:

- visualizar;
- confirmar;
- cancelar;
- expirar;
- mover mesa;
- bloquear/desbloquear;
- cortesia;
- no-show;
- concluir;
- reenviar pagamento;
- reenviar ingresso.

Toda ação crítica exige:

- motivo;
- usuário;
- timestamp;
- audit log.

---

# 52. RESERVA ADMINISTRATIVA PERSONALIZADA

ADM pode:

- mais de 8 pessoas;
- mais de duas mesas;
- união especial;
- cortesia;
- ajustes operacionais.

Mesmo assim:

validar conflito de mesa.

Nunca permitir overbooking silencioso.

---

# 53. MAPA ADM

Precisa selecionar evento.

Cada evento possui inventário próprio.

Exemplo:

```
Evento A / mesa 05 = confirmed
Evento B / mesa 05 = free
```

Estados nunca podem vazar entre eventos.

---

# 54. REALTIME

Usar Realtime para:

- hold criado;
- hold expirado;
- pagamento;
- bloqueio ADM;
- movimentação;
- check-in quando necessário.

Aplicar RLS.

Assinar apenas dados necessários.

Também implementar:

- refetch;
- reconexão;
- stale-state protection.

Nunca depender exclusivamente de WebSocket para integridade.

Banco é fonte de verdade.

---

# 55. SEGURANÇA RLS

Ativar RLS em tabelas expostas.

Público:

somente eventos públicos e disponibilidade sanitizada.

Cliente:

somente seus:

- dados;

- reservas;
- pagamentos resumidos;
- ingressos.

ADM:

RBAC.

Service role:

somente servidor.

---

# 56. RBAC

Papéis mínimos:

- owner;
- admin;
- manager;
- finance;
- box_office/check_in;
- viewer.

Aplicar autorização:

frontend + server + banco.

Nunca usar apenas “esconder botão”.

---

# 57. AUDIT LOG

Registrar:

- publicação de mapa;
- mudança de evento;
- preço;
- bloqueio;
- desbloqueio;
- movimento;
- confirmação;
- cancelamento;
- estorno;
- check-in revertido;
- reserva personalizada;
- conversão de lead WhatsApp;
- alteração de configuração;
- exportação de dados.

Audit log:

append-only.

---

# 58. ACCESSIBILITY

Corrigir:

- `<label>`;
- `required`;
- `aria-*`;
- nome acessível;
- foco;
- teclado;
- `autocomplete`;
- `inputmode`;
- contraste;
- reader support.

Mesa acessível deve comunicar:

- número;
- setor;
- capacidade;
- status.

Nunca depender só de cor.

---

# 59. MOBILE FIRST

Cliente fará reserva principalmente pelo celular.

Testar:

- iPhone viewport;
- Android viewport;
- tablet;
- desktop.

Mapa:

- zoom;
- pan;
- reset;
- minimapa;
- alternativa em lista.

---

# 60. EMPTY / ERROR / RETRY

Criar:

- loading;
- skeleton;
- empty state real;
- erro recuperável;
- offline;
- reconexão;
- retry.

Nunca mascarar ausência de dados causada por bug.

---

# 61. SEGURANÇA DO WHATSAPP

O número:

`+5511996963139`

é dado comercial público e pode aparecer no CTA.

Entretanto:

- não colocar número em 15 componentes;
- não duplicar template;
- não gerar URL manual em cada página.

Criar helper central:

```
buildWhatsAppReservationUrl(...)
```

Ele:

- lê configuração;
- normaliza;
- valida;
- gera mensagem;
- aplica `encodeURIComponent`;
- retorna URL.

Teste obrigatório:

URL final deve abrir:

`https://wa.me/5511996963139`

com mensagem preenchida corretamente.

---

# 62. WHATSAPP COMO CANAL OPERACIONAL

Usar também em:

- reserva personalizada;
- suporte da reserva;
- erro que realmente demande atendimento humano;
- detalhes da reserva, quando apropriado.

Não usar WhatsApp como substituto de um fluxo automatizável que deveria funcionar no sistema.

---

# 63. NOTIFICAÇÕES

Criar/usar outbox.

Eventos possíveis:

- hold_created;
- payment_pending;
- payment_approved;
- booking_confirmed;
- hold_expired;
- event_reminder;
- booking_cancelled;
- ticket_created.

Não acoplar envio ao commit da transação.

---

# 64. ENTIDADES DE DOMÍNIO

Adaptar ao banco existente, mas cobrir semanticamente:

- organizations
- profiles/customers
- staff_memberships
- venues
- areas
- map_templates
- map_versions
- tables
- table_positions
- table_join_rules
- events

- event_table_inventory
- bookings
- booking_tables
- booking_status_history
- terms_versions
- booking_terms_acceptances
- payments
- payment_events
- tickets
- checkins
- large_party_requests
- customer_notes
- customer_tags
- notifications_outbox
- idempotency_keys
- audit_logs
- background_jobs

Não renomear tabelas existentes sem necessidade.

---

# 65. MÁQUINA DE ESTADOS — BOOKING

Adaptar sem perder semântica:

- pending_payment
- confirmed
- expired
- cancelled_by_customer
- cancelled_by_admin
- completed
- no_show
- payment_review_required
- refund_pending
- refunded

Transições explícitas.

---

# 66. PAYMENT STATES

- created
- pending
- approved
- rejected
- cancelled
- expired
- refunded
- charged_back

# 67. TICKET STATES

- valid
- checked_in
- cancelled

---

# 68. IDEMPOTÊNCIA

Obrigatória para:

- criar booking;
- criar hold;
- pagamento;
- webhook;
- expiração;
- conversão ADM;
- confirmação manual;
- emissão de ticket;
- check-in.

---

# 69. APIs/RPCs

Adaptar arquitetura atual.

Domínio sensível deve rodar no backend.

RPC/service transacional para hold deve validar:

1. auth;
2. evento;
3. janela de venda;
4. mapa;
5. mesas;
6. disponibilidade;
7. capacidade;
8. limite self-service;
9. regra de união;
10. preço;
11. aceite;
12. idempotency key.

Só então gravar.

---

# 70. P0 — PARA OPERAÇÃO FUNCIONAL IMEDIATA

Prioridade absoluta:

1. eventos futuros na Landing;
2. Programação funcional;
3. Bom Jovi Cover navegável;
4. mapa real carregando;
5. seleção funcionando;
6. dados de cliente válidos;
7. criação real de reserva;
8. banco persistindo;
9. ADM recebendo;
10. status da mesa refletindo;
11. RLS segura;
12. ortografia UTF-8;
13. produção funcionando;
14. WhatsApp oficial configurado;
15. grupo >8 direcionado corretamente.

Não iniciar redesign enquanto esses itens falharem.

---

# 71. P1 — MOTOR COMPLETO DE RESERVAS

- conta;
- hold 3h;
- duas mesas;
- adjacências;
- Pix;
- webhook;
- ticket;
- área do cliente;
- CRM;
- reservas personalizadas;
- expiração;
- Realtime robusto.

---

# 72. P2 — OPERAÇÃO AVANÇADA

- editor completo de planta;
- versões;
- analytics avançado;
- funil;
- check-in;
- relatórios financeiros;
- automações;

- otimização de performance;
- CRM avançado.

---

# 73. MATRIZ DE REQUISITOS

Antes de implementar, criar:

`docs/master-requirements-matrix.md`

Colunas:

| ID | Requisito | P0/P1/P2 | Estado atual | Dependência | Implementação | Teste | Evidência | Status |

Nenhum P0 pode ficar apenas:

“planejado”.

---

# 74. CASOS DE BORDA

Testar obrigatoriamente:

1. dois clientes mesma mesa;
2. dois clientes com combinações sobrepostas;
3. duplo clique;
4. fechamento do browser;
5. hold expirando durante pagamento;
6. pagamento no último segundo;
7. webhook duplicado;
8. webhook atrasado;
9. webhook fora de ordem;
10. pagamento depois da expiração;
11. falha na segunda mesa;
12. admin bloqueia mesa com hold;
13. admin move reserva;
14. mapa alterado depois de venda;
15. preço alterado depois do hold;
16. CPF/e-mail/telefone duplicado;
17. sessão perdida;
18. grupo com 9 pessoas;
19. terceira mesa;
20. mesas 01 + 02;
21. evento cancelado;
22. evento adiado;
23. cliente tenta reserva alheia;
24. QR usado duas vezes;

25. provedor fora;
26. Realtime cai;
27. WhatsApp abre sem app instalado;
28. WhatsApp web em desktop;
29. telefone administrativo alterado;
30. template de WhatsApp com caracteres especiais.

---

# 75. TESTE PONTA A PONTA ATUAL

Provar:

```
casarao-360.vercel.app
↓
Bom Jovi Cover
↓
Garanta sua Experiência
↓
Mapa
↓
Mesa
↓
Login/Cadastro, se já exigido neste gate
↓
Solicitação/Hold
↓
Supabase
↓
ADM Reservas
↓
ADM Mapa
↓
Mesmo booking
↓
Mesma mesa
↓
Mesmo cliente
```

Sem isso:

P0 NÃO PASSOU.

---

# 76. TESTE WHATSAPP

Cenário:

Bom Jovi Cover.

Quantidade:

9 pessoas.

Resultado esperado:

- sistema não cria hold;
- sistema não bloqueia mesas;
- cria large_party_request;
- gera lead_code;
- CTA aparece;
- clique abre:

`https://wa.me/5511996963139`

Mensagem contém:

- Bom Jovi Cover;
- 12/09/2026;
- 9 pessoas;
- interesse;
- código de solicitação.

ADM recebe o lead.

PASS obrigatório.

---

# 77. TESTES GIVEN / WHEN / THEN

Exemplo:

### Cenário — duas mesas compatíveis

GIVEN: cliente autenticado; evento publicado; duas mesas livres; regra de união aprovada.

WHEN: selecionar grupo de 6; selecionar as duas mesas; confirmar.

THEN: um único booking; duas allocations; hold único; total único; mesas amarelas.

---

### Cenário — terceira mesa

GIVEN: grupo >8.

WHEN: cliente tenta reserva automática.

THEN: nenhum booking; nenhuma mesa presa; criar lead; CTA WhatsApp; abrir número oficial.

---

### Cenário — concorrência

GIVEN: uma mesa livre.

WHEN: dois clientes confirmam simultaneamente.

THEN: um vence; um recebe conflito; apenas um booking ativo.

---

# 78. TESTES TÉCNICOS

Executar:

```
npm run lint
npm run typecheck
npm run build
```

Se disponível:

```
npm test
npx playwright test
```

Adicionar testes de:

- domínio;
- RPC;
- RLS;
- concorrência;
- payment;
- webhook;
- accessibility;
- mobile;
- WhatsApp.

---

# 79. PERFORMANCE

Evitar N+1.

Programação:

query de eventos + disponibilidade agregada.

Mapa:

uma consulta de inventário do evento.

ADM:

agregações eficientes.

Índices:

baseados nas queries reais.

---

# 80. CORE WEB VITALS

Alvos quando tecnicamente alcançáveis:

- LCP p75 < 2,5 s
- INP < 200 ms
- CLS < 0,1

Não sacrificar integridade transacional por performance cosmética.

---

# 81. LGPD

Aplicar:

- minimização;
- finalidade;
- acesso restrito;
- mascaramento;
- auditoria;
- consentimento de marketing separado;
- direito do titular;
- política de retenção;
- exclusão conforme obrigação legal.

---

# 82. PRODUÇÃO E MOCK

Production:

dados reais.

Proibir fallback silencioso para mock.

Se Supabase falhar:

mostrar erro controlado.

Não trocar automaticamente por dados falsos.

---

# 83. MIGRATION STRATEGY

Sempre:

1. auditar schema;
2. snapshot;
3. escrever migration;
4. validar local;
5. revisar SQL;
6. aplicar;
7. validar dados;
8. executar testes;
9. documentar.

Backfill:

idempotente.

---

# 84. FEATURE FLAGS

Quando necessário:

- new_customer_account;
- multi_table_booking;
- dynamic_payment;
- map_versions;
- new_admin;
- checkin.

Flags permitem rollout.

Não criar dívida permanente.

---

# 85. DEPLOY

Depois dos gates:

1. commit;
2. push;

3. Vercel Production;
4. confirmar SHA;
5. smoke test no domínio real;
6. verificar logs;
7. confirmar Supabase;
8. confirmar auth;
9. confirmar reserva;
10. confirmar ADM.

“Deployment Ready” não é prova de funcionamento.

---

# 86. ROLLBACK

Registrar:

- deployment anterior;
- SHA;
- migrations;
- mudanças de banco;
- feature flags.

Rollback de código não deve presumir rollback destrutivo de banco.

Migrations precisam ser forward-compatible.

---

# 87. NÃO DECLARAR FALSO POSITIVO

Não significa “concluído”:

- componente criado;
- migration escrita;
- build passou;
- tabela existe;
- RPC existe;
- constraint existe;
- teste isolado passa.

Conclusão exige:

**runtime real.**

---

# 88. RELATÓRIO DE RECONCILIAÇÃO DO MAPA

Antes de publicar novo inventário:

informar:

- mesas atuais;
- novas mesas declaradas;
- correspondências;
- códigos órfãos;
- reservas impactadas;
- capacidade antes;
- capacidade depois;
- pendências 26–28;
- decisão.

Não inventar.

---

# 89. ORTOGRAFIA E LINGUAGEM

Cliente:

- Garanta sua Experiência
- Programação e Experiências
- Próximas Experiências
- Experiências Realizadas
- Horário em breve
- Mesa disponível
- Mesas disponíveis
- Aguardando pagamento
- Reserva confirmada
- Reserva expirada
- Falar no WhatsApp para reserva personalizada

ADM:

- Painel Executivo
- Central de Reservas
- Mapa Inteligente do Salão
- Lista Inteligente de Espera
- Centro de Apoio Operacional
- Visão de Crescimento

---

# 90. FONTES OFICIAIS DE ENGENHARIA

Aplicar princípios atuais de:

PostgreSQL: `https://www.postgresql.org/docs/current/explicit-locking.html`

Usar locks de linha/transações para concorrência real.

Supabase RLS: `https://supabase.com/docs/guides/database/postgres/row-level-security`

RLS em schemas expostos.

Supabase Realtime: `https://supabase.com/docs/guides/realtime/postgres-changes`

Avaliar também Broadcast conforme arquitetura e escala.

Mercado Pago Webhooks: `https://www.mercadopago.com.br/developers/pt/docs/your-integrations/notifications/webhooks`

Validar origem da notificação e assinatura secreta quando aplicável.

OpenTable: combinações de mesas, mapas e dining areas como referência de produto.

Sympla: check-in e experiência de evento como referência operacional.

Não copiar branding.

---

# 91. SEGURANÇA DE PAGAMENTO

Nunca:

- confiar em callback de frontend;
- colocar token do provedor no cliente;
- aceitar valor informado pelo browser;
- confirmar reserva antes do servidor validar pagamento.

---

# 92. SECURITY DEFINER

Auditar functions privilegiadas.

Preferir:

```
SET search_path = ''
```

com referências schema-qualified.

Revisar:

`GRANT EXECUTE`.

Não liberar funções privilegiadas desnecessariamente para `anon`.

# 93. RATE LIMIT

Aplicar para:

- login;
- cadastro;
- criação de hold;
- lead WhatsApp;
- pagamento;
- check-in;
- endpoints sensíveis.

---

# 94. OBSERVABILIDADE

Registrar sem PII:

- falhas de reserva;
- conflito;
- expiração;
- webhook;
- pagamento;
- auth;
- realtime;
- lead WhatsApp;
- erro de mapa.

Nunca logar CPF completo.

---

# 95. BACKUP E INCIDENTE

Documentar:

- backup;
- restore;
- rollback;
- incidente de pagamento;
- reserva duplicada;
- indisponibilidade do Supabase;
- indisponibilidade Vercel;
- indisponibilidade Mercado Pago.

---

# 96. CRITÉRIOS DE P0

Para P0 PASS:

- agenda real aparece;
- próximo evento correto;
- evento abre;
- mapa real abre;
- seleção funciona;
- reserva é persistida;
- ADM recebe;
- mapa ADM reflete;
- concorrência protege;
- RLS protege;
- UTF-8 correto;
- mobile funcional;
- Production testada;
- WhatsApp oficial funciona para >8;
- nenhum dado falso em produção.

---

# 97. CRITÉRIOS DE RESERVA MULTIMESA

PASS somente se:

- duas mesas compatíveis;
- uma reserva;
- duas alocações;
- transação atômica;
- total único;
- hold único;
- pagamento único;
- CRM único;
- ticket único;
- nenhuma terceira mesa self-service.

---

# 98. CRITÉRIO DO WHATSAPP

PASS somente se:

- número vem da configuração;
- default atual = `+5511996963139`;
- link final = `https://wa.me/5511996963139`;
- template configurável;
- mensagem preenchida;
- lead salvo;
- nenhuma mesa bloqueada;

- ADM consegue converter;
- alteração futura do número não exige deploy.

---

# 99. GATES DE EXECUÇÃO

## GATE 0 — Descoberta

Auditar:

- código;
- DB;
- migrations;
- produção;
- eventos;
- mapa;
- auth;
- pagamentos;
- ADM;
- CRM;
- tests.

Entregar baseline.

---

## GATE 1 — P0 FUNCIONAL

Corrigir:

- agenda;
- Landing;
- evento;
- mapa;
- reserva;
- banco;
- ADM;
- RLS;
- WhatsApp;
- UTF-8.

Somente avançar com E2E aprovado.

---

## GATE 2 — DOMÍNIO CANÔNICO

Implementar:

- map versions;
- inventory;

- bookings;
- allocations;
- states;
- adjacency;
- idempotency;
- audit.

---

## GATE 3 — CONTA DO CLIENTE

Implementar:

- auth;
- perfil;
- minhas reservas;
- meus ingressos.

---

## GATE 4 — MULTIMESA

Implementar:

- 2 mesas;
- compatibilidade;
- roxo;
- atomicidade;
- WhatsApp >8.

---

## GATE 5 — PAGAMENTO

Implementar:

- hold 3h;
- Pix;
- webhook;
- conciliação;
- ticket.

---

## GATE 6 — ADM / CRM

Implementar:

- reservas;
- mapa;
- clientes;
- pagamentos;

- leads;
- relatórios.

---

## GATE 7 — CHECK-IN

Implementar:

- QR;
- portaria;
- no-show;
- realtime.

---

## GATE 8 — HARDENING

Executar:

- segurança;
- RLS;
- RBAC;
- accessibility;
- performance;
- observabilidade;
- backup;
- E2E completo.

---

# 100. RELATÓRIO FINAL

Criar:

`docs/casarao-360-master-release-report.md`

Para cada Gate:

- estado inicial;
- causa raiz;
- arquivos alterados;
- migrations;
- SQL;
- decisões;
- testes;
- evidências;
- riscos;
- pendências.

---

# 101. MATRIZ FINAL

Apresentar:

| Gate | Status | Evidência | Bloqueio |
|---|---|---|---|

Estados:

- PASS
- FAIL
- BLOCKED

---

# 102. STATUS FINAL PERMITIDO

Somente:

- BLOQUEADO
- P0 OPERACIONAL
- PRONTO PARA HOMOLOGAÇÃO
- PRONTO PARA PRODUÇÃO
- PRODUÇÃO HOMOLOGADA

Nunca:

“100% pronto”

se algum P0 estiver sem prova.

---

# 103. RESULTADO OBRIGATÓRIO PARA A APRESENTAÇÃO

O fluxo mínimo precisa ser real:

```
Cliente
↓
casarao-360.vercel.app
↓
Bom Jovi Cover
↓
Mapa
↓
Mesa disponível
↓
```

```
Reserva
↓
Banco
↓
ADM
↓
Central de Reservas
↓
Mapa ADM
```

E para 9 pessoas:

```
Cliente
↓
Bom Jovi Cover
↓
9 pessoas
↓
Self-service bloqueado corretamente
↓
large_party_request
↓
Código de lead
↓
Falar no WhatsApp
↓
+55 (11) 99696-3139
↓
ADM
↓
Reserva Personalizada
```

Esses dois circuitos precisam existir sem quebrar um ao outro.

---

# 104. REGRA FINAL

Preserve tudo que já funciona.

Corrija primeiro o coração operacional.

Depois evolua.

Não reescreva por vaidade arquitetural.

Não esconda erros.

Não invente dados.

Não contorne segurança.

Não substitua persistência real por mock.

Não confunda frontend bonito com produto pronto.

O Casarão 360 só estará realmente concluído quando:

**cliente, operação, banco e administração estiverem olhando para a mesma reserva, o mesmo evento, as mesmas mesas, o mesmo pagamento e a mesma fonte de verdade.**